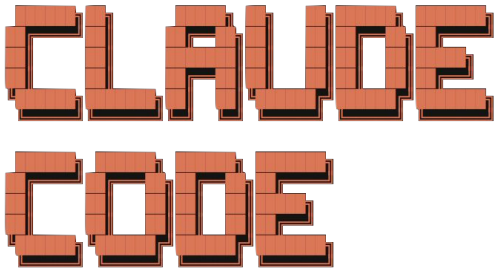
Introduction to LLM Agents

From language models to agentic AI

LLM Agents





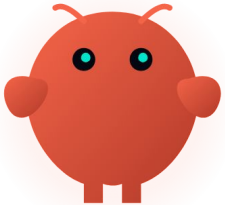
ChatGPT



OpenAI
Codex



Claude



OpenClaw



Sure! How about we try making some Korean BBQ Nachos? Here's the recipe:

Ingredients:

- 1 lb ground beef or pork
- 1/2 cup Korean BBQ sauce (you can find this at most grocery stores or Asian markets)
- 1 bag of tortilla chips
- 2 cups shredded Monterey Jack cheese
- 1/2 cup chopped green onions
- 1/4 cup chopped cilantro
- 1/4 cup chopped jalapeños
- 1/4 cup sour cream
- Lime wedges for garnish

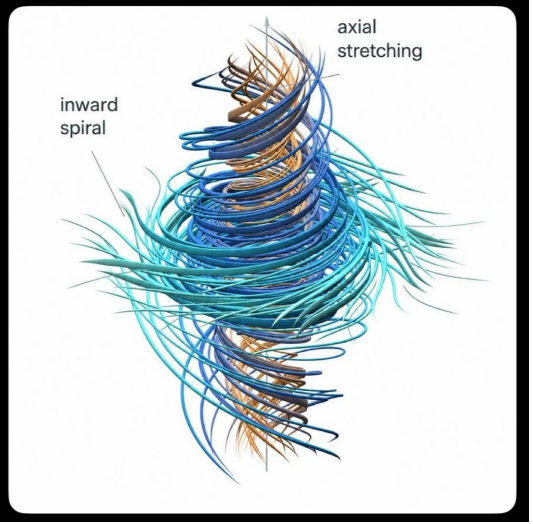
3 year later...

OpenAI @OpenAI · 36m

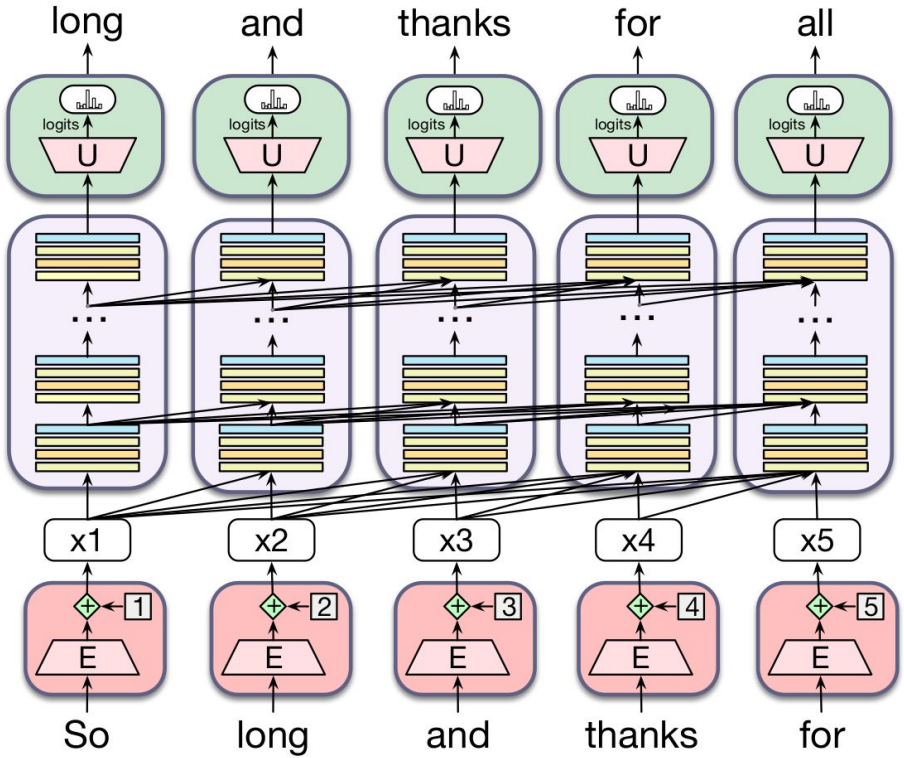
We're sharing a solution to the Navier-Stokes Millennium Prize Problem, one of the deepest problems at the frontier of mathematics.

The proof was produced by a group of agents, using an OpenAI next-generation model significantly more capable than GPT-6 Astra.

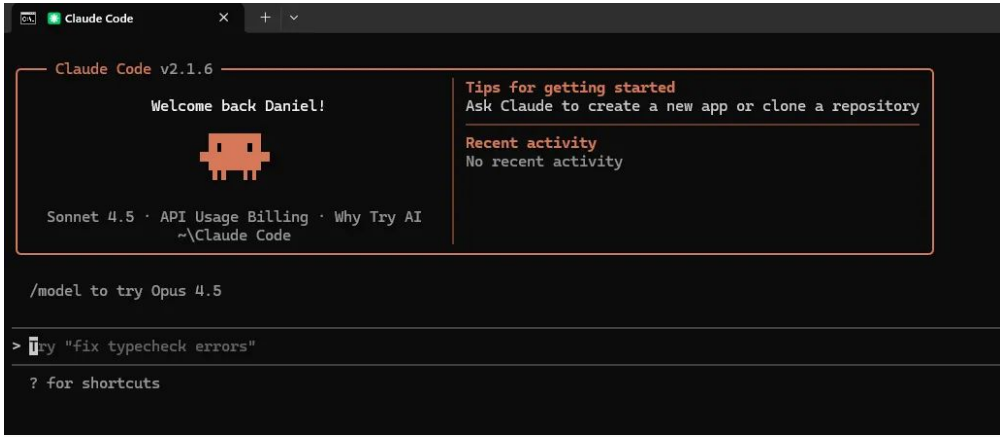
The problem concerns whether the description of smooth three-dimensional fluid motion modeled by the Navier-Stokes equations can break down. It has remained unresolved for roughly 90 years.

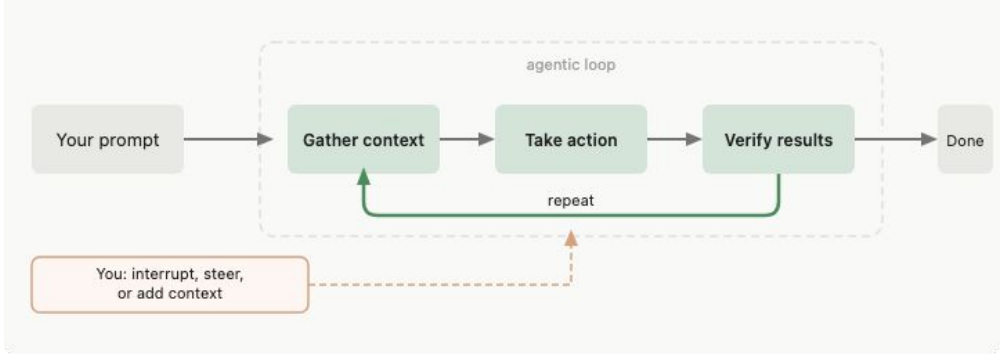


How to build GPT from scratch



How to build CLAUDE CODE from scratch





How to build CLAUDE CODE from scratch

1. Scale the LLM pretraining phrase
2. Scale the LLM generation phase (test-time scaling)
3. Enable the LLM to use files and tools
4. Wrap the LLM in an agentic framework

How to build CLAUDE CODE from scratch

1. **Scale the LLM pretraining phrase**
2. Scale the LLM generation phase (test-time scaling)
3. Enable the LLM to use files and tools
4. Wrap the LLM in an agentic framework

LLM Scaling Laws

CSC3044S AI Systems

Lecturer: Dr Francois Meyer

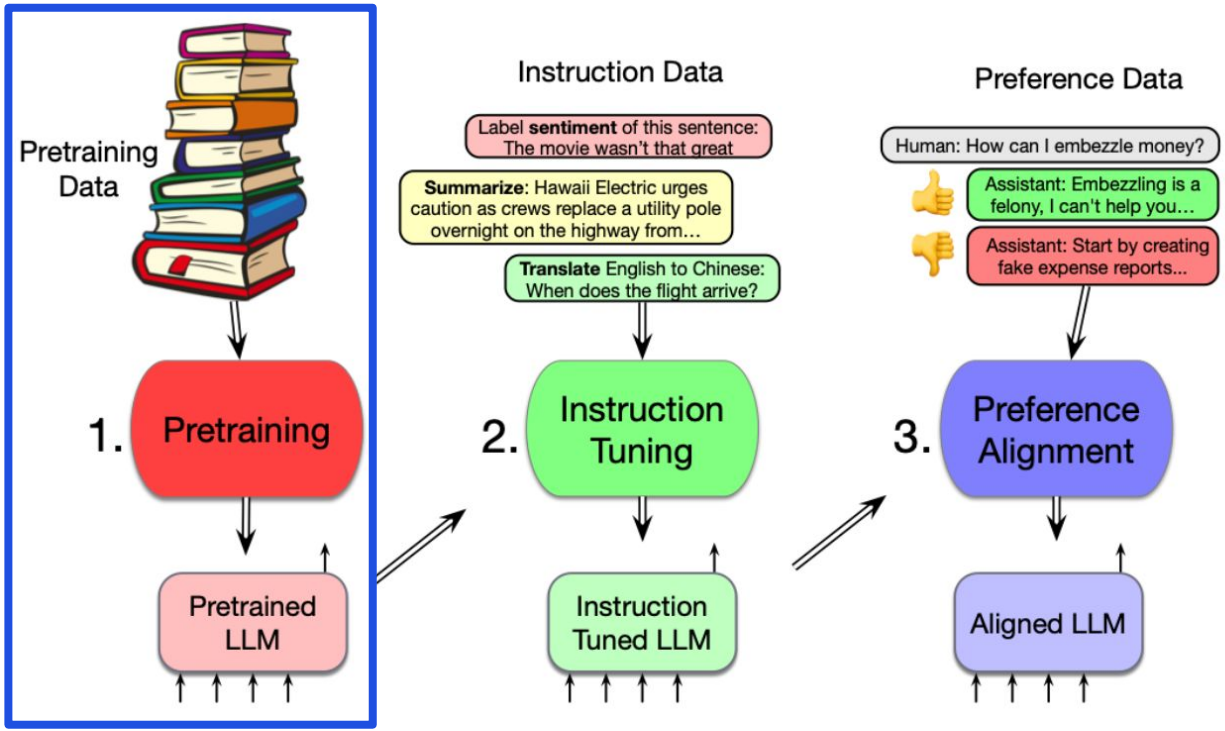


Computer Science Department
University of Cape Town

LLM Agents Topic 1
5 October 2026

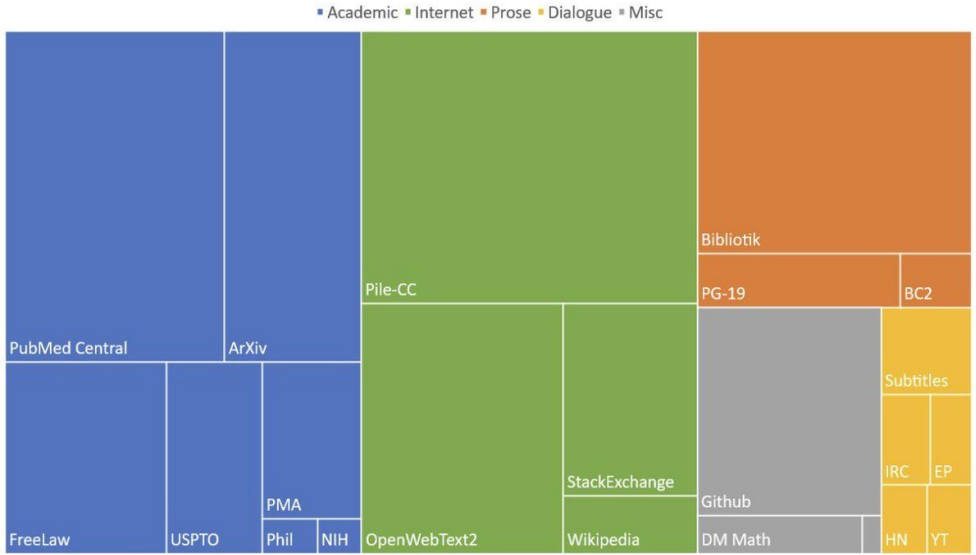
Based on Xiao & Zhu, *Foundations of LLMs*

Recap: LLM Training



Pretraining

- Pretraining is the most computationally expensive part of developing LLMs.
- LLMs are now typically pretrained on trillions tokens of text, as scaling pretraining improves performance.



Llama 3.1 (405B parameters)

126

blocks
stacked transformer blocks

15T+

tokens
of text seen during training

16,000

GPUs
NVIDIA H100s in parallel

80 days

30.84M GPU-hours
of compute on the 405B alone

Kimi K3 (2.8 trillion parameters)

93

blocks

stacked transformer blocks

2.8T

parameters

weights learned during training

104B

active parameters

used on any single token

1M

tokens

of context the model can read at once

How to scale pretraining?

- Scale **data**: larger pretraining datasets
- Scale **model size**: increase the number of parameters
- Scale **compute**: train for more GPU hours

But these factors are dependent on each other...

More data requires bigger models and longer training.

Bigger models need more data to avoid over-fitting.

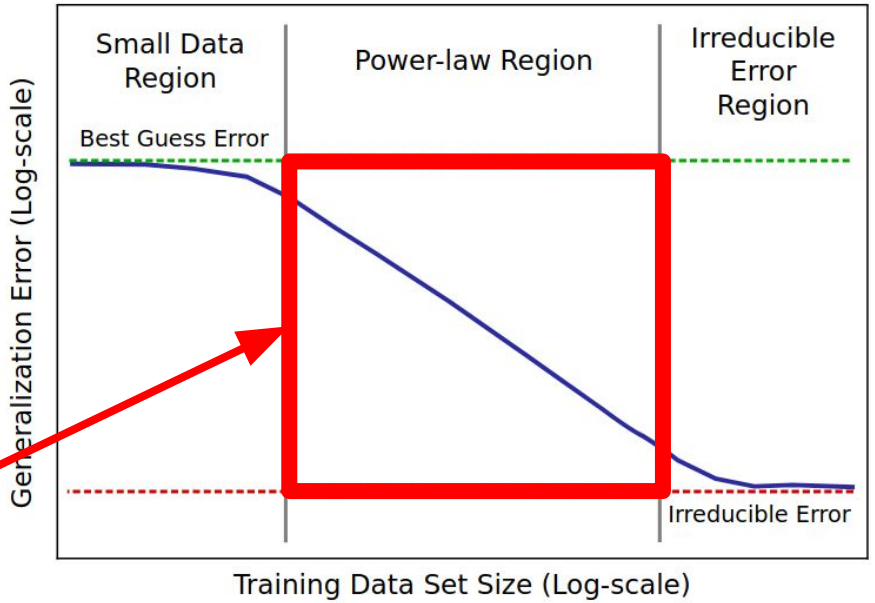
Scaling Laws

LLM scaling laws describe the relationship between **performance** (measured as test set loss) and **model size**, **pretraining data size**, and **pretraining compute**.

As pretraining data grows...

- Initially, slow improvements.
- At some stage, performance improves rapidly.
- Later, improvement slows again.

Scaling laws apply here.



Scaling Laws

Let x be the variable of interest (e.g. number of model parameters) and $L(x)$ be the loss of the model given x (e.g. cross-entropy loss on test data).

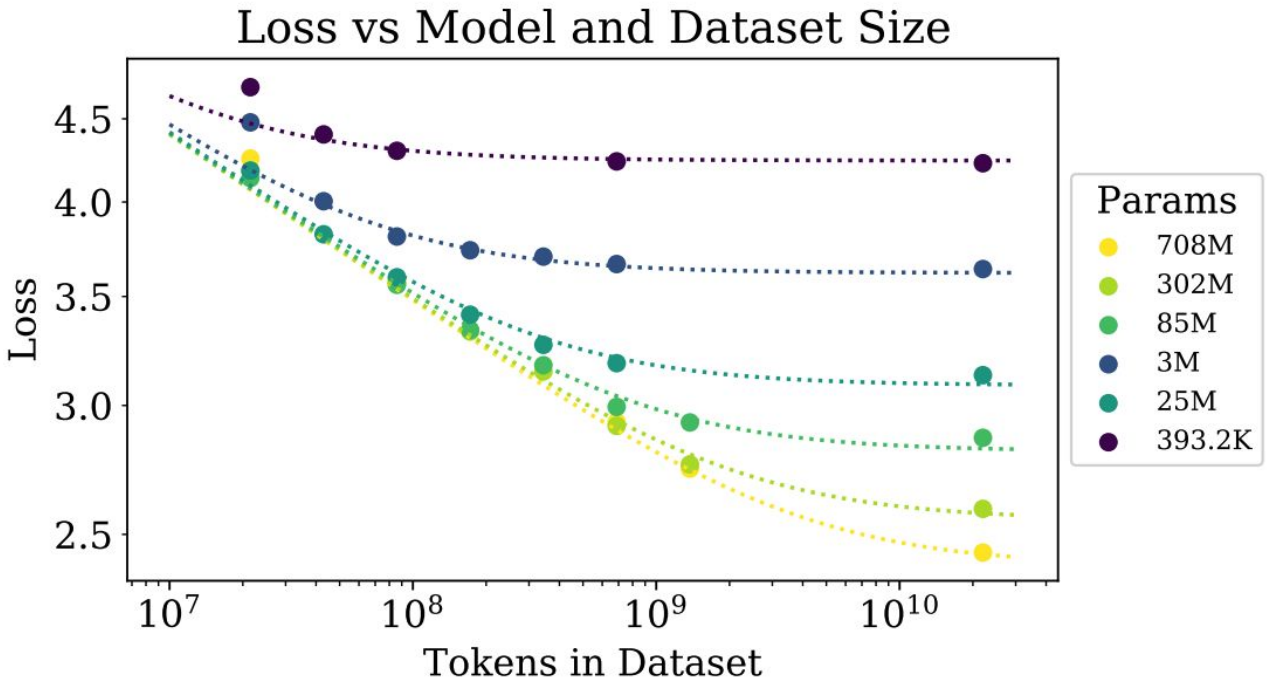
We can model this relationship with a power law:

$$L(x) = ax^b$$

where a and b are parameters that are estimated empirically by training many models of different sizes and tracking loss.

For LLMs, the resulting parameters $a > 0$ and $b < 0$.

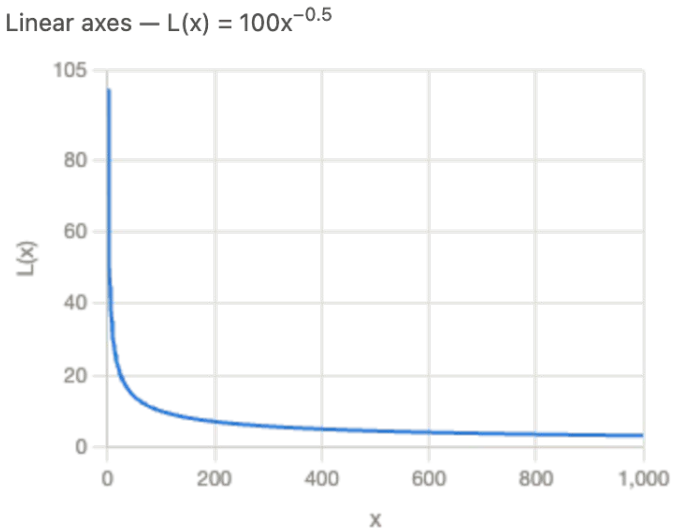
Scaling Laws



Scaling Laws

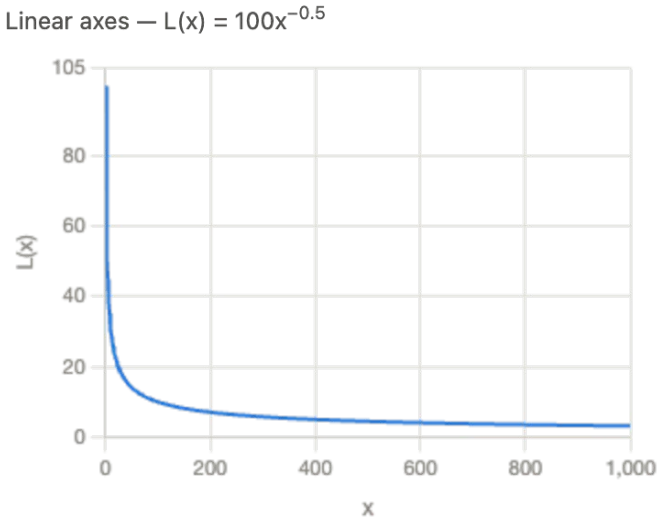
$$L(x) = ax^b \text{ with } b < 0$$

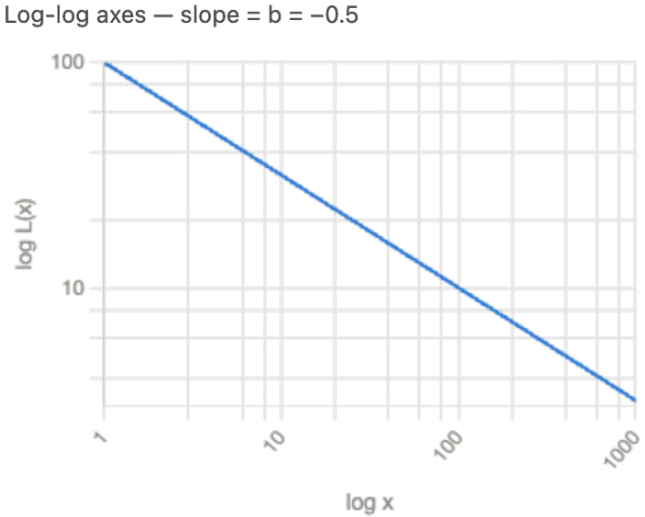
- Loss falls as x (parameters or data) grows.
- Multiplying x by 10 <u>always</u> multiplies loss by 10^b (same proportion), whether from 1M to 10M parameters or from 10B to 100B.
- But each 10× scaling costs 10× more than last and absolute gains become smaller!
- That is why it feels like scaling diminishes. Constant improvements at increasing cost!



Scaling Laws

$$L(x) = ax^b \quad \Leftrightarrow \quad \log L(x) = \log(a) + b\log(x)$$





E.g. if $b = -0.05$, every 10× scaling cuts loss by 11%. But 11% of a small loss is worth much less than 11% of a large one.

Scaling Laws

Suppose N denotes model size (number of parameters) and D denotes dataset size (number of tokens).

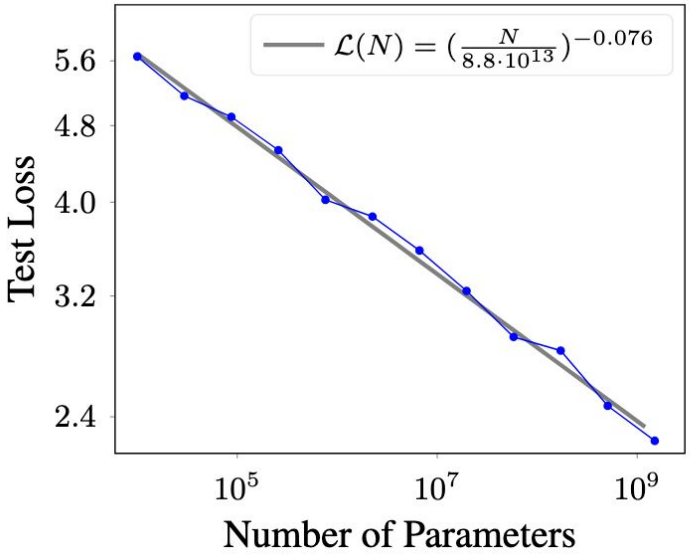
Example: Kaplan et al. (2020) trained Transformer LMs from 768M–1.5B parameters with datasets from 22M–23B tokens and showed that performance improves predictably as a power law of N or D.

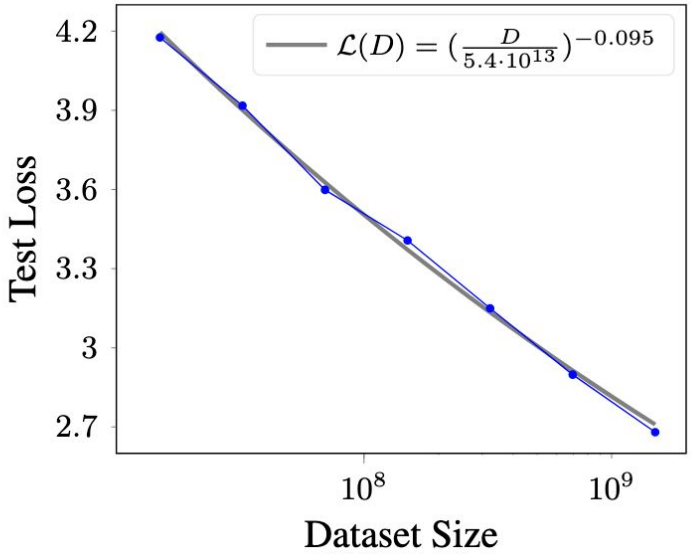
$$L(N) = \left(\frac{N}{8.8\times10^{13}}\right)^{-0.076}$$

$$L(D) = \left(\frac{D}{5.4\times10^{13}}\right)^{-0.095}$$

Scaling Laws

Performance (test set loss) improves predictably as a power law of N or D.





Improved Scaling Laws

The loss can never be zero – model can never perfectly predict language modelling test set, because language is inherently stochastic.

$$L(x) = ax^b + E$$

where E is an irreducible error term, which is the minimum loss attainable (given an infinitely large model trained on infinite data).

A scaling law can be expressed in terms of model size and data size scaling:

$$L(N, D) = aN^b + cD^d + E$$

Improved Scaling Laws

Example: Chinchilla Scaling Laws (Hoffman et al., 2022) found by training over 400 models from 70M–16B parameters with datasets from 5B–500B tokens.

$$L(x) = aN^b + cD^d + E$$

$$\mathcal{L}(N, D) = \underbrace{\frac{406.4}{N^{0.34}}}_{\text{model scaling}} + \underbrace{\frac{410.7}{D^{0.28}}}_{\text{dataset scaling}} + \underbrace{1.69}_{\text{irreducible error}}$$

Scaling Laws

NOTE: These “laws” are not universal like laws of mathematics/physics – they are regularities based on empirical observations. Too many confounding factors…

- **Hyperparameters** e.g. learning rate, batch size & optimisers e.g. SGD, Adam
- **Tokenisation** decisions
- **Data** quality & repetition (some scaling laws assume no repeating data)

Alternative scaling laws have been developed to take these into account.

Another problem: if we infer scaling laws based on experiments with small models, they are not guaranteed to be accurate for larger models.

Scaling Laws

Scaling laws are useful: they can guide decisions during LLM development.

- Given X available compute, how should we prioritise **data vs model size**?
- Given Y amount of data, **how big** should our model be?
- If we scale our model/data, **how much improvement** should we expect?

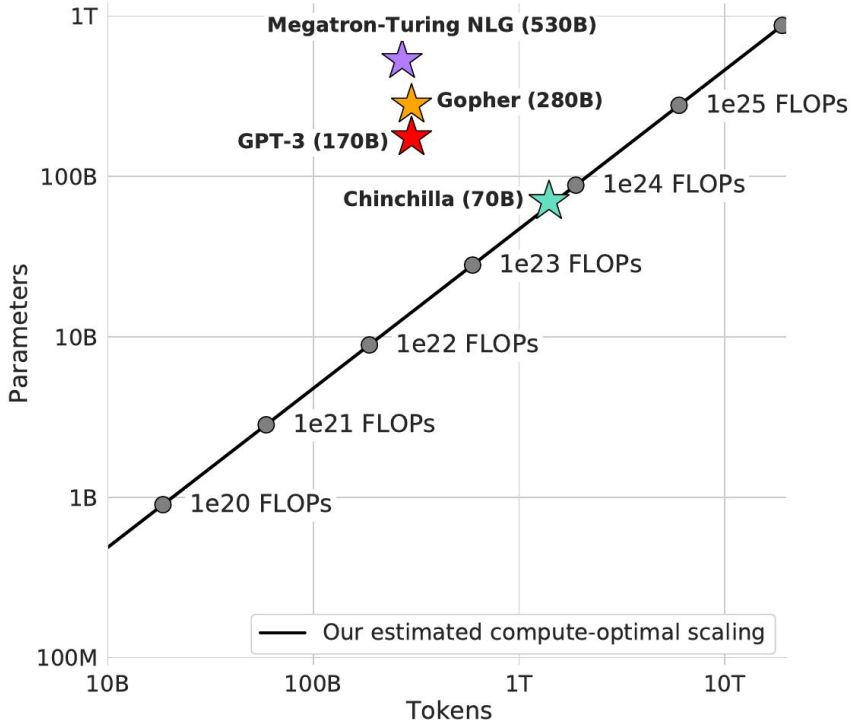
Even OpenAI and Anthropic want to optimise resource usage, and scaling laws allows us to reduce reliance on trial-and-error.

Scaling Laws Findings

Takeaway #1: Loss follows power laws in model size, dataset size, and compute.

Scale together for optimal performance!

Scale parameters and tokens in equal proportions (if you double model size, double the training tokens too).



We can use scaling laws for more concrete guidance...

Scaling Laws Findings

Let N denote model size (number of parameters), D dataset size (number of tokens), and C compute budget (number of training FLOPs).

Given a fixed compute budget C, how do we decide on N and D?

Training FLOPs can be computed from model and training set size as:

$$C \approx 6 \times N \times D$$

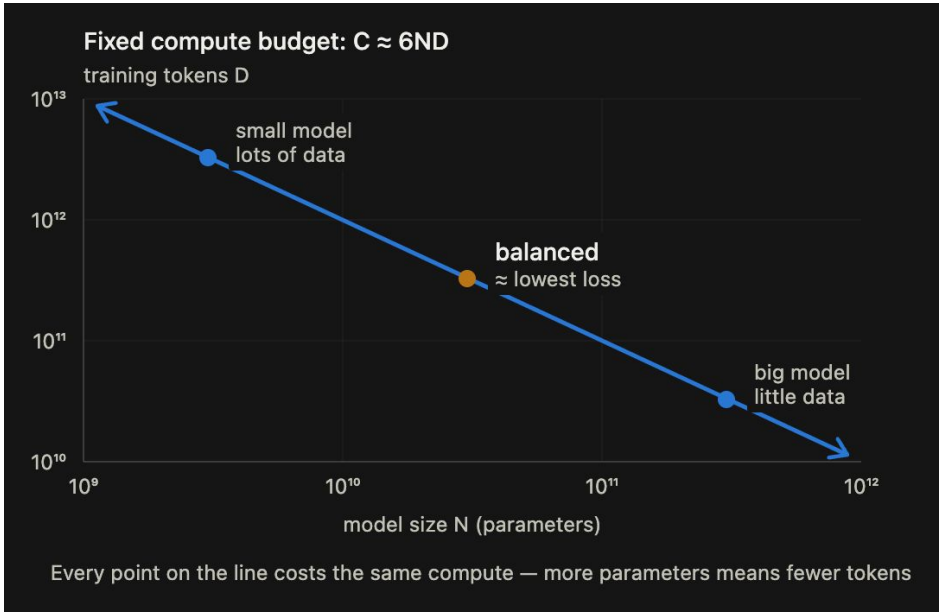
parameters tokens

($\sim 2ND$ for forward passes and $\sim 4ND$ for backward passes)

Optimising for Compute

Given a fixed compute budget, we have to make a tradeoff between model size and training data size.

- If we increase one, that means decreasing the other.
- **How should we split available compute between data and model size?**



- Find the parameter–token ratio that minimises the loss!

Optimising for Compute

Example: Chinchilla Scaling Laws (Hoffman et al., 2022)

$$\mathcal{L}(N, D) = \underbrace{\frac{406.4}{N^{0.34}}}_{\text{model scaling}} + \underbrace{\frac{410.7}{D^{0.28}}}_{\text{dataset scaling}} + \underbrace{1.69}_{\text{irreducible error}}$$

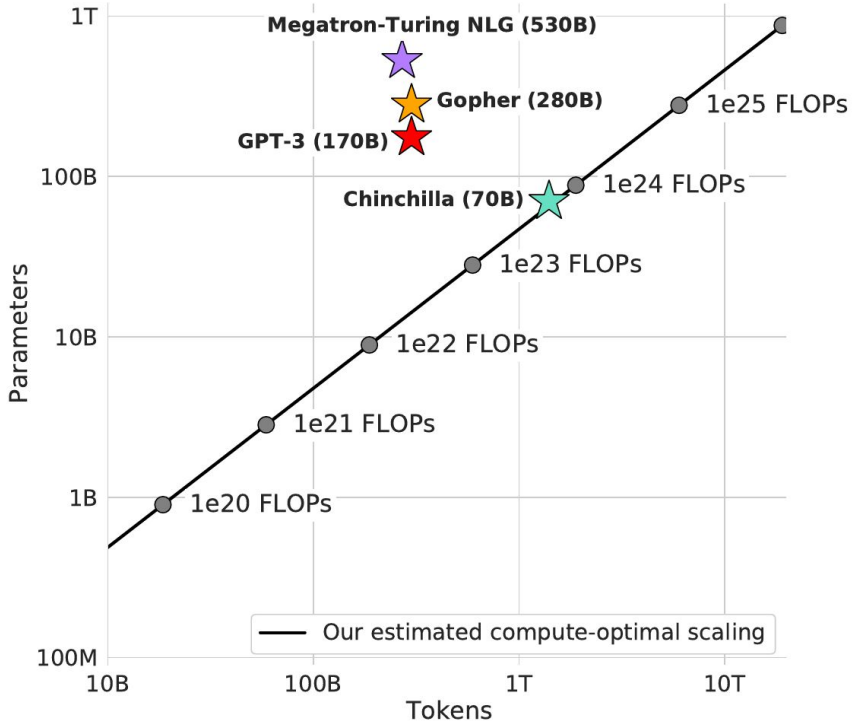
Scaling laws can be used to derive performance-optimal parameter–token ratios under a fixed compute budget.

For these laws, it was shown be ~**1:20**.

Optimising for Compute

How do we choose N and D?

- **20 training tokens per parameter.**
- E.g. on a budget of 10^{23} FLOPs, train a 70B model on 1.4T tokens.
- **This showed that most LLMs were undertrained!**
- E.g. Chinchilla outperformed Gopher under same compute budget, despite being 4× smaller.



Applications of Scaling Laws

$$C \approx 6 \times N \times D$$

parameters tokens

Suppose we assume the optimal parameter : token ratio is 20:1.

Here at UCT we have A100 GPUs, which do about **10^{14} FLOPs per second**.

(a) You get one A100 for a one-hour tutorial slot. What should you train?

(b) You get one A100 for 24 hours. What should you train?

Applications of Scaling Laws

$C = 6ND$,

Optimal N:D ratio is 1:20, so substitute $20N$ for D.

$C = 6N(20N) = 120N^2$

Solve for N:

$$N = \sqrt{\frac{C}{120}}$$

(a) One A100, one hour

$$C = 3600\ \text{s} \times 10^{14}\ \text{FLOP/s} = 3.6 \times 10^{17}\ \text{FLOPs}$$

$$N = \sqrt{C/120} = \sqrt{3.0 \times 10^{15}} \approx 55\text{M parameters}$$

$$D = 20N \approx 1.1\text{B tokens}$$

(b) One A100, 24 hours

$$C = 86{,}400\ \text{s} \times 10^{14}\ \text{FLOP/s} = 8.6 \times 10^{18}\ \text{FLOPs}$$

$$N = \sqrt{C/120} = \sqrt{7.2 \times 10^{16}} \approx 270\text{M parameters}$$

$$D = 20N \approx 5.4\text{B tokens}$$

Optimising for Data

If data is the limiting factor, scaling laws show which model size optimises loss.

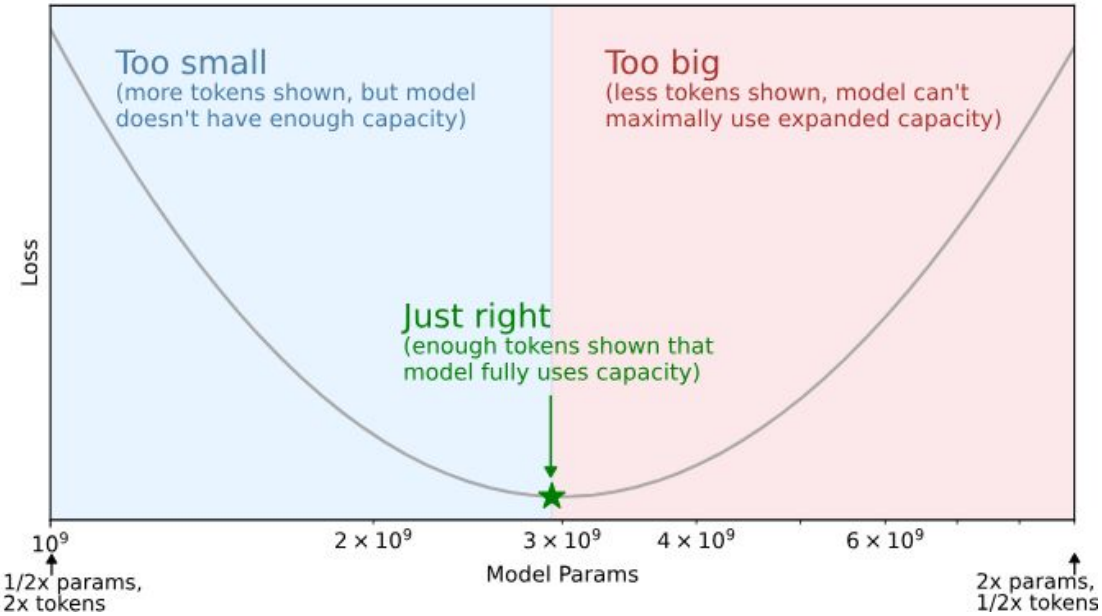


Image from https://irhum.github.io/blog/chinchilla/.

Optimising for Data

For the MzansiLM project, we wanted to train a multilingual LM on all 11 official written languages of South Africa. Our **main constraint is data, not compute**.

We first created **MzansiText**, a web-scraped dataset for all 11 languages:

Language	Train Tokens
Afrikaans	2,475,913,822
English	740,994,679
isiNdebele	818,549
Sepedi (Northern Sotho)	6,697,358
Sesotho	97,558,939
siSwati	1,932,989
Setswana	10,082,930
Xitsonga	3,013,408
Tshivenda	1,852,481
isiXhosa	152,212,403
isiZulu	320,224,015
Total	3,811,301,573

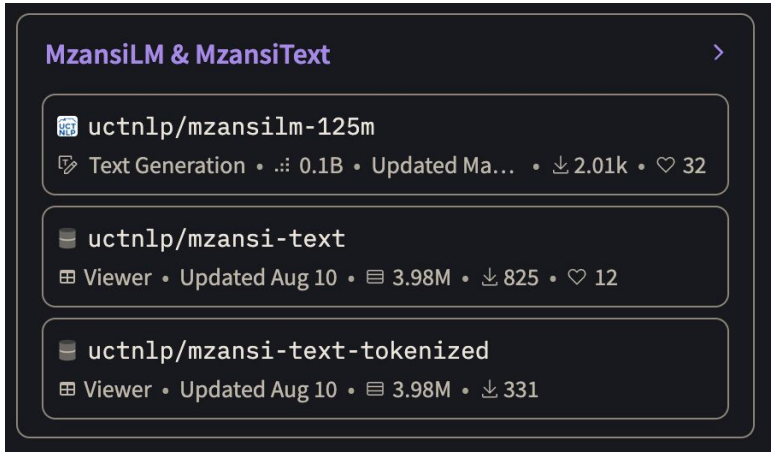
We downsample Afrikaans to balance training across languages and end up with a training set of **2.5B tokens**.

How big should **MzansiLM** be?

Based on 1:20 parameter-token ratio:

2.5B / 20 = 125M parameters

Optimising for Data



Training a larger model wastes compute on parameters the data can't support.

Training a smaller model leaves performance on the table.

NOTE: We are ignoring the fact that we train our model for multiple epochs! Repeating data gives diminishing returns, which complicate scaling laws.

The scaling laws we looked at (e.g. Kaplan et al., Chinchilla) assume no data is repeated, so our calculations here are approximations.

Scaling Laws Findings

Takeaway #2: Architectural details matter less than sheer scale!

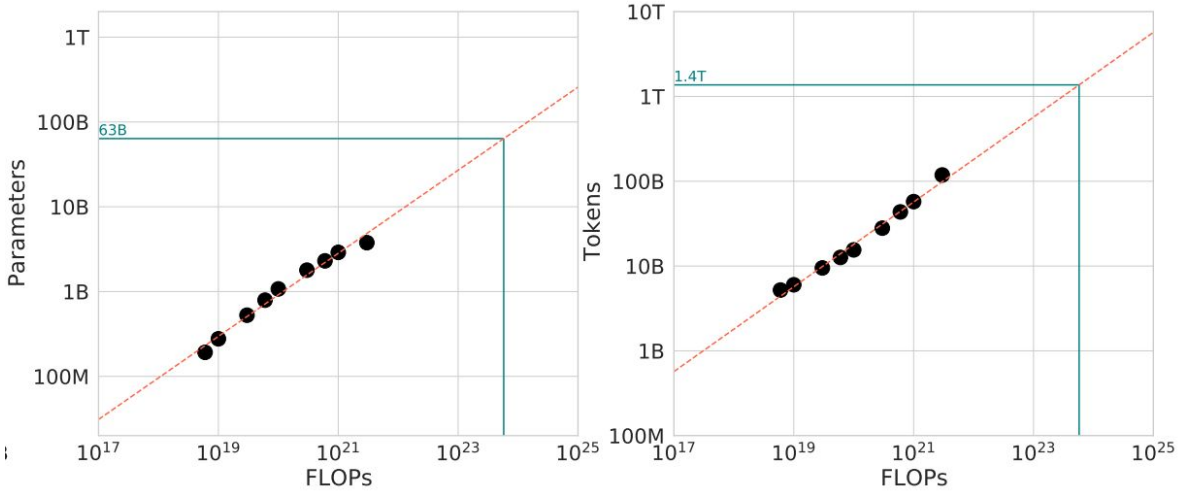
For Transformers, model width (dimensions) and depth (number of layers) have small effects relative to how much compute, data and parameters are used.



Applications of Scaling Laws

If you are working for a company with lots of data but limited compute, how do you know if it's worth scaling further? Is it worth the cost?

If we scale up further, how much loss improvement should we expect?



Scaling Laws Findings

Takeaway #3: These laws predict loss, not downstream capability.

**Downstream* refers to specific, end-user tasks or applications that use a pretrained LLM to solve a real-world problem.

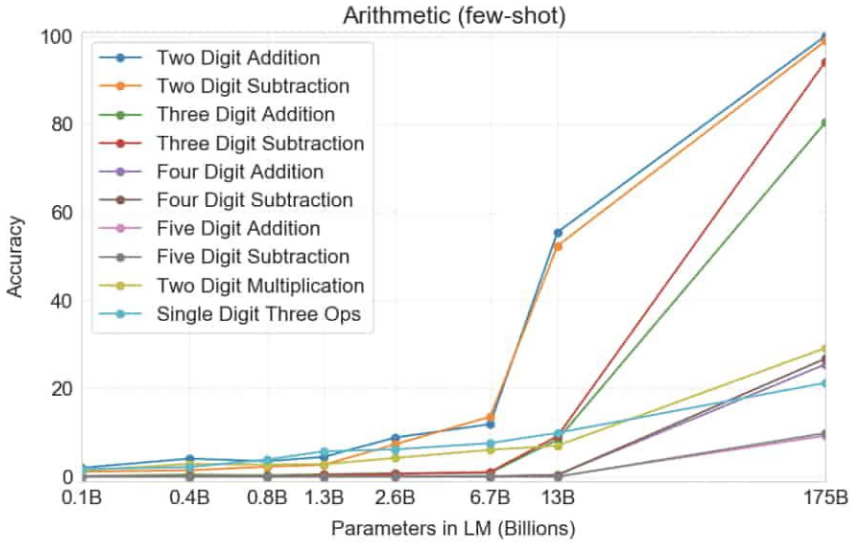
Scaling laws do not tell us about downstream performance on real world tasks.

Pretraining loss is correlated with performance for some tasks, but not reliably or as predictably...

Emergence

Emergent capabilities: skills or behaviours that do not exist in smaller LMs but appear suddenly and unpredictably when a model reaches a threshold of scale, parameter count, and training data.

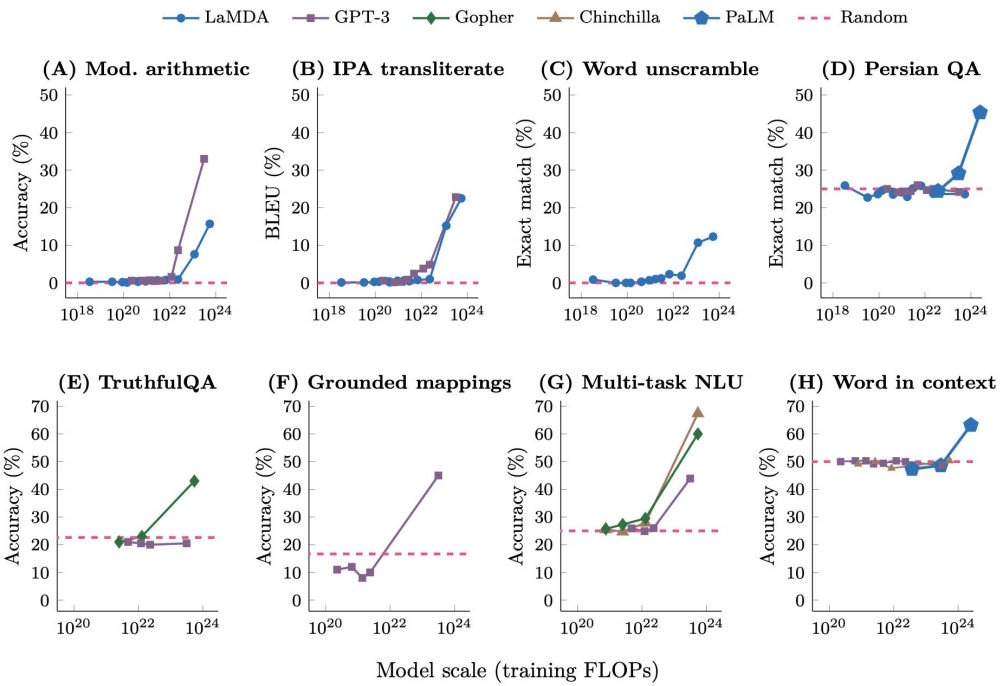
For example:



Emergence

Different tasks obey different laws.

- Some capabilities only emerge after a given scale of model / data size / compute.
- Some skills exhibit **phase transitions:** sudden, discontinuous jumps in performance.



Wei et al., "Emergent Abilities of Large Language Models." TMLR, 2022

Reading

- **These slides are the main resource to study for the exam.**
- **Prescribed reading:** Section 2.2.4 in Foundations of Large Language Models by Xiao & Zhu: https://arxiv.org/pdf/2501.09223

- Optional reading if you want to learn about this in more depth: https://lilianweng.github.io/posts/2026-06-24-scaling-laws/

References

- Kaplan et al. “Scaling Laws for Neural Language Models.” arXiv preprint arXiv:2001.08361, 2020.
- Hoffmann et al. “Training Compute-Optimal Large Language Models.” NeurIPS, 2022.